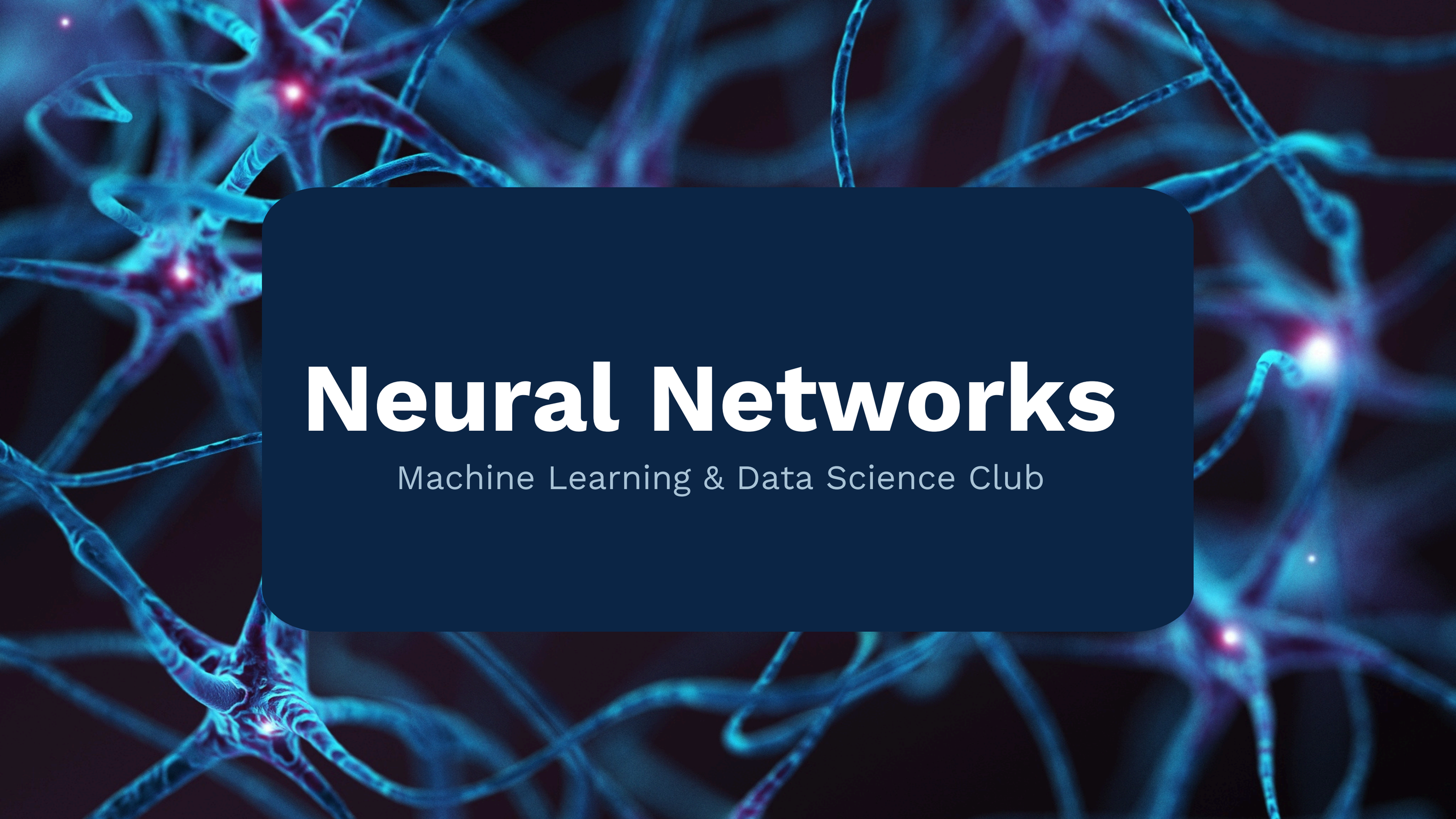
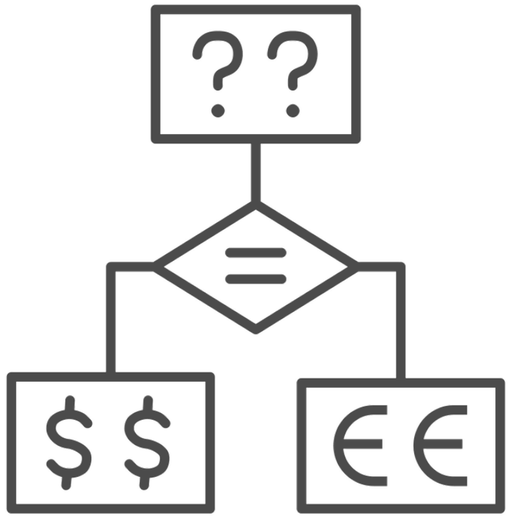


Neural Networks

Machine Learning & Data Science Club

“Traditional” or “Classical” ML

Tree-Based Methods





Support Vector Machines





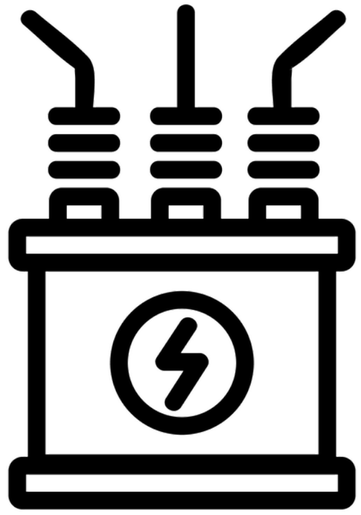
Linear/Logistic Regression

Deep Learning

Neural Networks



Transformers



So far, every problem has been a table

lead_time	deposit_type	adr	prev_cancels	is_canceled
342	No Deposit	0.00	0	0
7	No Deposit	75.00	0	0
85	Non Refund	82.00	1	1
211	No Deposit	103.50	0	1
…	…	…	…	…

rows = examples
columns = features

Someone had to decide **which columns exist.**

Classify: Blåhaj or real shark?



Blåhaj



Shark

Let’s pick some features...

feature	Blåhaj	shark
blue-gray?	✓	✓
has fins?	✓	✓
has teeth?	✓	✓
white belly?	✓	✓
underwater?	✗	✓
fluffy?	?	?

What actually separates a shark from a Blåhaj?

Things like fluffy, stitched, or looks like fabric are easy to see, but really hard to write as a rule.

What the computer actually sees



57 110 122	38 89 101	50 102 119	67 123 143	58 115 131	64 120 131	79 129 142	80 128 143
61 115 127	34 92 103	45 81 91	64 76 88	61 69 83	89 101 114	81 135 142	72 125 143
36 83 97	49 62 70	69 46 44	59 39 27	60 48 39	77 49 46	104 97 100	88 136 152
48 63 77	66 39 46	54 54 53	150 154 154	42 46 44	29 29 29	48 29 35	103 118 130
41 42 53	59 42 45	47 56 56	139 138 135	49 44 48	37 41 39	41 38 38	110 109 123
28 58 72	71 56 58	56 38 39	28 28 24	37 35 37	44 37 35	90 71 72	109 139 149
46 91 108	61 101 111	89 83 94	79 59 64	82 59 63	87 72 75	71 92 95	58 107 116
41 75 93	73 118 135	66 116 133	55 92 107	63 96 107	74 121 134	34 86 99	20 57 70

each square is one pixel = 3 numbers (red, green, blue), each 0–255

(t**his is the eye)**

But doesn’t this also have a tabular representation?

150,528

224 × 224 pixels × 3 colors
= numbers in one small photo

px1_R	px1_G	px1_B	…	px39,085_R	px39,085_G	px39,085_B	…	px50,176_B
192	190	191	…	49	44	48	…	128

Technically, yes.
But does this actually tell us anything?

The problem with pixels as features



original
pixel #39,085 = (49, 44, 48):
the eye



moved 10 px right
pixel #39,085 = (37, 89, 103):
fabric

are these both still Blåhajs?

Alone, a pixel means nothing.
“Pixel #39,085 is dark” says nothing about sharks.

Shift the image and the numbers change.
A 10-pixel shift noticeably changed about half of the 150,528 values.

The meaning is in how pixels relate to each other.
(think edges, textures, shapes)

A different approach

classical ML  → a person designs the features → model → **Blåhaj**

deep learning  → **model learns the features and the prediction** → **Blåhaj**

But how do we do this?

Features built from features



pixels



edges



textures



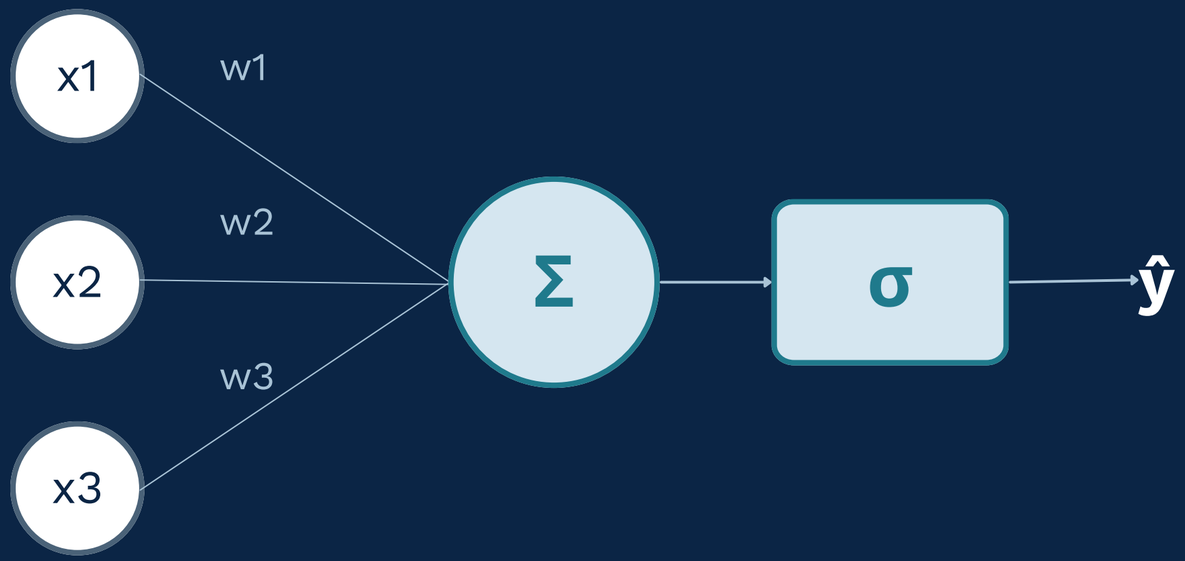
parts



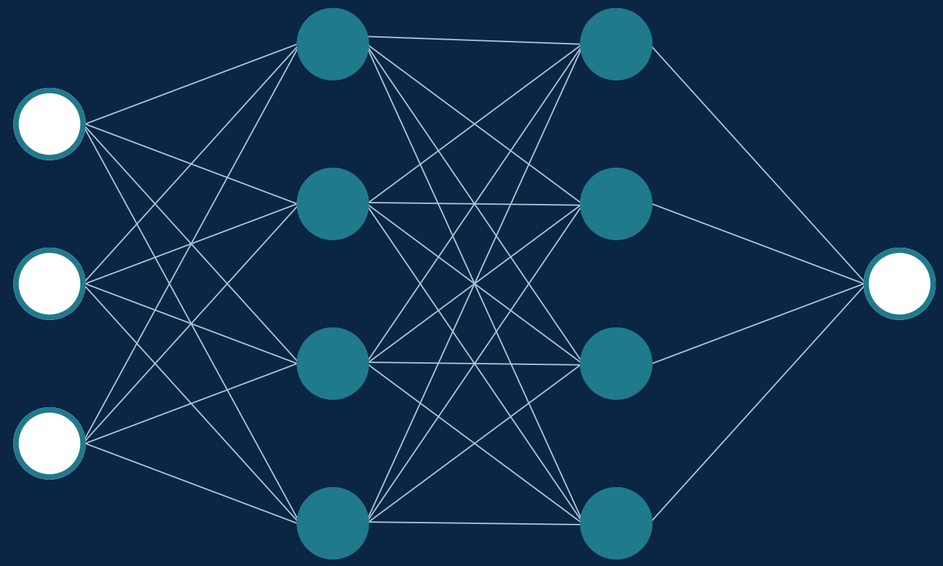
Blåhaj

Each layer combines the last layer's features to get a bigger, more meaningful picture.

Stacking what we already know



One neuron = a weighted sum + an activation function. With a sigmoid, that's exactly logistic regression (hidden neurons usually use ReLU instead).



You stack a bunch of them!
You go from raw data → a prediction

The anatomy of a neural network

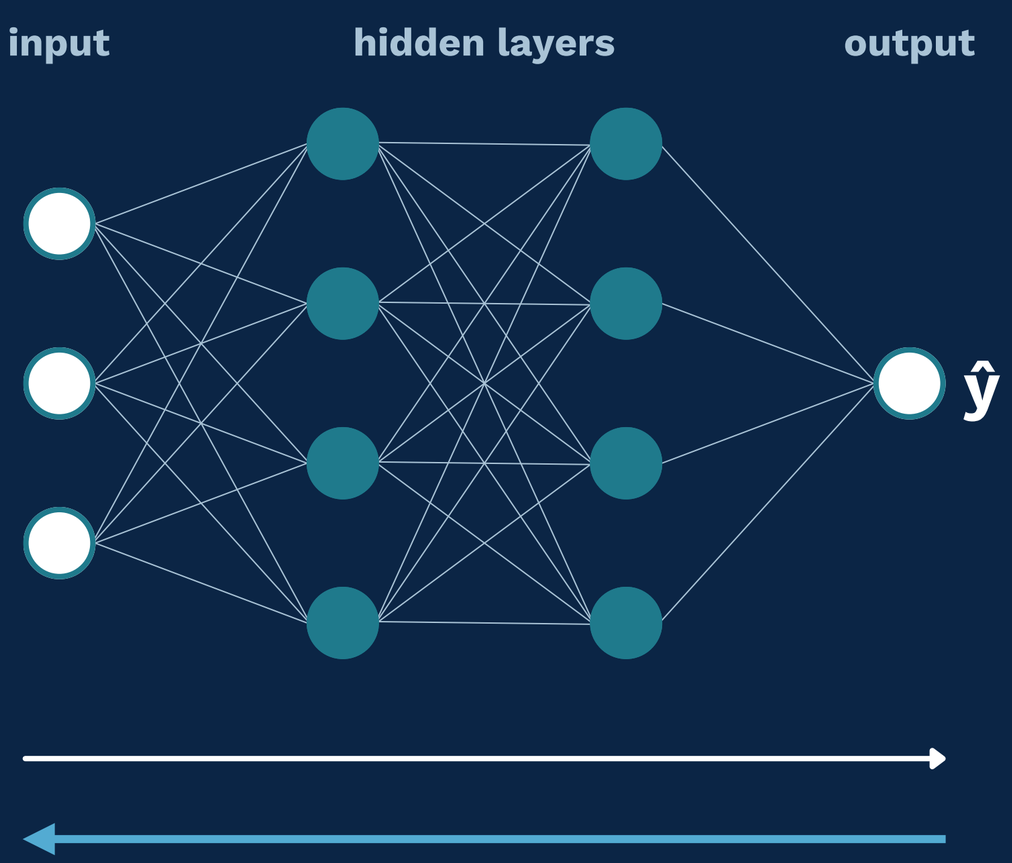
Input layer: the raw data, one neuron per feature (for an image, one per pixel).

Hidden layers: each neuron takes a weighted sum of the layer before and applies an activation. Patterns build and compound (edges → shapes → parts).

Output layer: the prediction (e.g. a probability for each class).

→ **Forward pass** (white arrow): data flows left to right, layer by layer, to make a prediction.

← **Backpropagation** (blue arrow): compare the prediction to the right answer, send the error backward, and nudge every weight to shrink it. Repeat many times.



The many types of neural networks

Feedforward Neural Networks (FNN / MLP)

Best used for: Simple classification, regression, and structured/tabular data (such as spam detection or risk assessment).

Convolutional Neural Networks (CNNs)

Best used for: Image recognition, computer vision, facial recognition, and medical image analysis

Recurrent Neural Networks (RNNs)

Best used for: Time-series prediction, speech recognition, and basic sequential data (mostly replaced by transformers for language)

Transformers

Best used for: Language (chatbots like ChatGPT and Claude, translation), and increasingly images too

...

The big tradeoff

Pro: they can model complex relationships

Neural nets capture high-dimensional, non-linear patterns: thousands of inputs interacting in ways no straight line can describe (like which pixels, together, make a fin).

Direct consequence: we lose interpretability

The same structure that makes them powerful makes it hard to say why a prediction was made. There's no single number that says "this feature matters this much."

Compare: linear regression

One coefficient per feature: "+1 unit of x_1 changes the prediction by β_1." Easy to read and explain, but it can only model straight-line effects.

So where does the interpretability go?

It comes down to how the network is built. Let's look at the math.

Why can't we just read the weights?

Linear regression: one number per feature

$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots$

β_1 is the effect of x_1, and it's the same no matter what the other features are.

A neural network: functions inside functions

$\hat{y} = W_3 \cdot \sigma(W_2 \cdot \sigma(W_1 x + b_1) + b_2) + b_3$

Every input gets mixed into every neuron, bent by σ (e.g. ReLU), then mixed and bent again in the next layer.

So an input's effect isn't one number

Because σ is non-linear, how much x_1 matters depends on the values of all the other inputs. The same pixel can push toward "shark" in one drawing and away from it in another.

...and it's spread across a lot of weights

Even a small net for 28×28 doodles (784 → 128 → 64 → 4) has ~109,000 parameters. Concepts get spread across many neurons, and one neuron often responds to several unrelated things.

Uses, pros & cons

Where they're used

Image recognition, speech-to-text, translation, chatbots, medical imaging, recommendation systems, self-driving cars

Pros

Learn their own features from raw data · capture complex, non-linear patterns · keep improving with more data · works for images, audio, and text

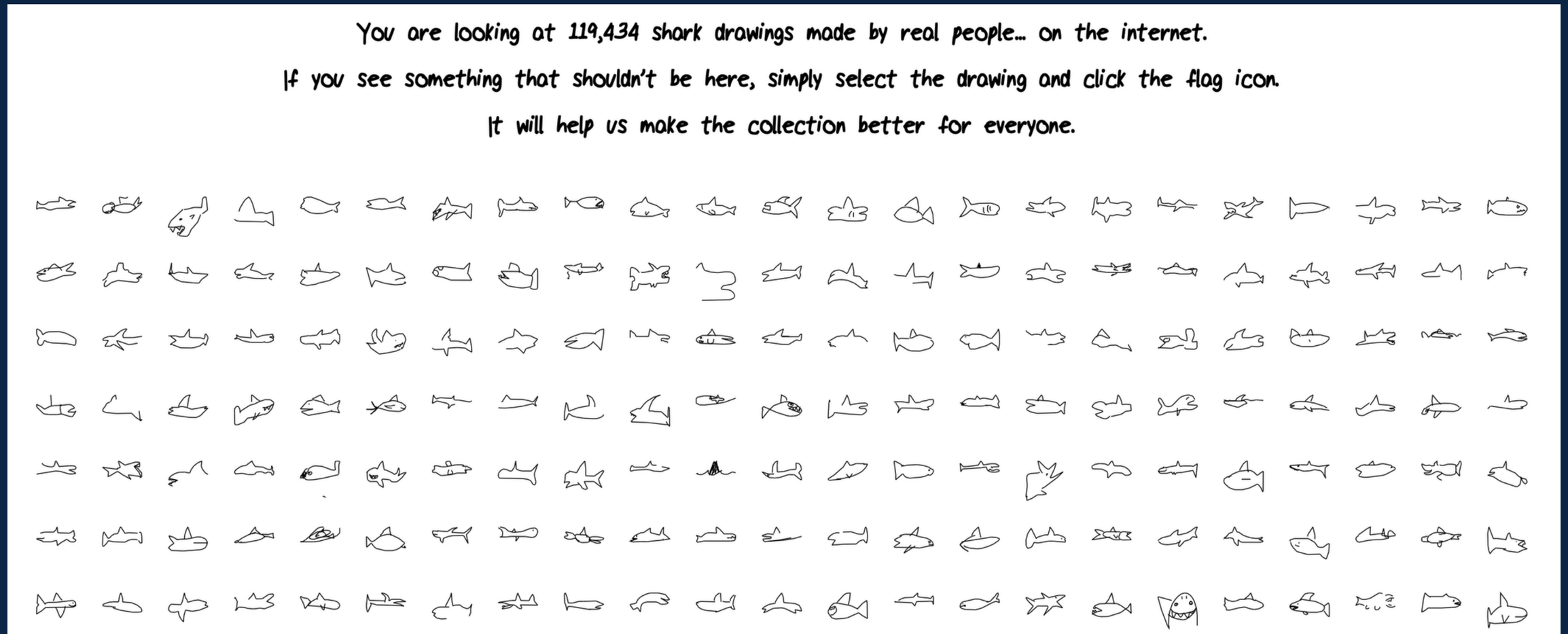
Cons

Hard to interpret · data-hungry (need a lot of data, it overfits on small datasets) · expensive to train (often need GPUs) · lots of design choices to tune (layers, neurons, learning rate…)

A note on tabular data

Neural nets can be (and are) used on tables, since they capture complex relationships there too. But they won't always beat simpler methods: on typical tabular data, gradient-boosted trees (like XGBoost) often match or outperform them, and train faster.

Our data: Google **Quick, Draw!**



The task: doodle → sea animal

Classify drawings into 4 sea animals (including sharks!)

Each doodle is a 28 × 28 grayscale image: 784 numbers, one per pixel, each 0–255 for how much ink is there.

Why is this hard for traditional ML?

Same problem as the Blåhaj: one pixel means nothing on its own. People draw at different sizes, angles, and styles, so the meaning is in how pixels relate to each other.

Our model: a feed-forward neural network

784 pixels in → hidden layers that learn strokes and shapes → 4 outputs, one probability per animal.

Why a feed-forward net can handle this

Doodles are small, black-and-white, and scaled to the same frame: far simpler than a 150,528-number photo. (CNNs, which know which pixels are neighbors, would do even better.)